Block-Diagonal Jacobian of the LDG Residual in Exasim

1. Summary

For the LDG path in Exasim, the exact block-diagonal Jacobian of

$$J(U) = \frac{\partial R(U)}{\partial U}$$

with U the global vector of the degrees of freedom of u_h, is most naturally implemented as one dense element block per element:

$$J_{\text{diag}} = \text{diag}(J_{11}, J_{22}, \ldots, J_{N_e N_e}),$$

where each block

$$J_{KK} = \frac{\partial R_K}{\partial U_K}$$

has size $(npe \cdot ncu) \times (npe \cdot ncu)$, with R_K the residual rows belonging to element K.

In the current Exasim backend, the cleanest exact implementation is not to derive and assemble a separate sparse Jacobian by hand, but to reuse the existing residual tangent path:

- `backend/Discretization/residual.cpp`
- `backend/Discretization/qresidual.cpp`
- `backend/Discretization/uresidual.cpp`
- `backend/Discretization/getuhat.cpp`
- `backend/Discretization/matvec.cpp`

Specifically:

- if Enzyme AD is enabled, assemble each diagonal block column-by-column using `dResidual(...)`
- otherwise, assemble each block column-by-column using `MatVec(...)` with finite differences

That approach automatically includes all LDG chain-rule contributions through:

- `GetUhat(...)`
- `GetQ(...)`
- `GetW(...)`
- `GetAv(...)`
- `RuElem(...)`
- `RuFace(...)`

2. Assumptions

- U contains only the DOFs of u_h, not the auxiliary variables q, w, or face trace `uh`.
- The residual $R(U)$ is exactly the one returned by `Residual(sol, res, app, driver_abi, master, mesh, tmp, common, handle, backend)` in `backend/Discretization/residual.cpp`.
- Therefore q, `uh`, `wdg`, and optional AV-related fields are implicit functions of U, because `Residual(...)` recomputes them internally before assembling `res.Ru`.
- A "block" means one element block of size:

$$n_{\text{loc}} = npe \cdot ncu.$$

- The desired matrix is the exact block-diagonal of the full LDG Jacobian, not a simplified "volume-only" or "element-local-only" approximation.
- I am focusing on the LDG residual path, not the HDG Schur-complement system.
- I assume the block-diagonal Jacobian may later be inverted batched, for example for a block-Jacobi preconditioner.

3. Design reasoning

The key point is that the residual returned by `Residual(...)` is not just a function of the element-local u. It is a composite map:

$$U \longmapsto \hat{U}(U),\ Q(U),\ W(U),\ AV(U) \longmapsto R(U).$$

So the exact diagonal block must include all chain-rule paths created by the residual pipeline.

A. Residual structure used by `Residual(...)`

In serial, the LDG residual pipeline is:

1. `GetUhat(...)`
2. `GetQ(...)`
3. `GetW(...)` if `common.ncw > 0`
4. `GetAv(...)` if AV is active and unfrozen
5. `RuElem(...)`
6. `RuFace(...)`
7. `PutFaceNodes(...)`
8. final scaling by `-1` and optionally `1/common.dtfactor`

This is in `backend/Discretization/residual.cpp`.

So if I define the raw assembled residual before the final sign/time scaling by $\widetilde{R}(U)$, then the residual returned by `Residual(...)` is

$$R(U) = \alpha\, \widetilde{R}(U),$$

with

$$\alpha = \begin{cases} -1, & \text{steady case,} \\ -\dfrac{1}{\texttt{common.dtfactor}}, & \text{time-dependent case.} \end{cases}$$

Therefore each diagonal block is

$$J_{KK} = \alpha\, \frac{\partial \widetilde{R}_K}{\partial U_K}.$$

That final scalar factor must be included.

B. Element-block definition

Let $U_K \in \mathbb{R}^{n_{\mathrm{loc}}}$ be the u_h DOFs of element K, with

$$n_{\mathrm{loc}} = npe \cdot ncu.$$

Let $R_K(U)$ be the residual entries in `res.Ru` associated with element K. Then

$$J_{KK} = \frac{\partial R_K}{\partial U_K}.$$

This is the exact diagonal block. It keeps all dependence of the residual rows of element K on the DOFs of the same element K, while discarding all derivatives with respect to DOFs from other elements.

C. Chain rule through the LDG pipeline

The raw residual for element K can be viewed schematically as

$$\widetilde{R}_K(U) = \widetilde{R}_K^{\mathrm{elem}}\big(U_K, Q_K(U), W_K(U), AV_K(U)\big) + \sum_{f \subset \partial K} \widetilde{R}_{Kf}^{\mathrm{face}}\big(U, Q(U), W(U), \hat{U}(U)\big).$$

Hence the exact diagonal block is

$$\frac{\partial \widetilde{R}_K}{\partial U_K} = \frac{\partial \widetilde{R}_K^{\mathrm{elem}}}{\partial U_K} + \frac{\partial \widetilde{R}_K^{\mathrm{elem}}}{\partial Q_K}\frac{\partial Q_K}{\partial U_K} + \frac{\partial \widetilde{R}_K^{\mathrm{elem}}}{\partial W_K}\frac{\partial W_K}{\partial U_K} + \frac{\partial \widetilde{R}_K^{\mathrm{elem}}}{\partial AV_K}\frac{\partial AV_K}{\partial U_K} + \sum_{f \subset \partial K} \frac{\partial \widetilde{R}_{Kf}^{\mathrm{face}}}{\partial U_K}^{\mathrm{total}}.$$

The face contribution expands further into:

$$\frac{\partial \widetilde{R}_{Kf}^{\mathrm{face}}}{\partial U_K}^{\mathrm{total}} = \frac{\partial \widetilde{R}_{Kf}^{\mathrm{face}}}{\partial U_{K,f}} + \frac{\partial \widetilde{R}_{Kf}^{\mathrm{face}}}{\partial Q_{K,f}}\frac{\partial Q_{K,f}}{\partial U_K} + \frac{\partial \widetilde{R}_{Kf}^{\mathrm{face}}}{\partial W_{K,f}}\frac{\partial W_{K,f}}{\partial U_K} + \frac{\partial \widetilde{R}_{Kf}^{\mathrm{face}}}{\partial \hat{U}_f}\frac{\partial \hat{U}_f}{\partial U_K}.$$

This is the correct mathematical content of the diagonal block in Exasim.

D. Contribution from `GetUhat(...)`

`GetUhat(...)` is in `backend/Discretization/getuhat.cpp`.

For LDG:

- on interior faces, `sol.uh` is copied from one side's u
- on boundary faces, `sol.uh` is produced by `UbouDriver(...)`

Therefore the derivative

$$\frac{\partial \hat{U}_f}{\partial U_K}$$

can be nonzero for:

- the element that owns the copied interior-face trace
- any boundary face of element K

This derivative is implemented in the tangent path by:

- `GetdUhat(...)`
- `dUhatBlock(...)`

in the same file.

That means the exact diagonal block must include the way a perturbation in U_K changes `uh` first, and then changes face residuals and q-recovery through that `uh`.

E. Contribution from `GetQ(...)`

`GetQ(...)` is in `backend/Discretization/residual.cpp`.

It computes q by:

- `RqElem(...)`
- `RqFace(...)`
- `PutFaceNodes(...)`
- `ApplyMinv(...)`
- `ArrayInsert(...)` into `sol.udg`

From the implementation in `backend/Discretization/qresidual.cpp`, the elementwise operator is

$$Q_K = M_K^{-1}\left(C_K U_K - E_K \hat{U}_K + S_K^q\right),$$

where:

- M_K is the local mass matrix inverse applied through `ApplyMinv(...)`

- C_K is the element gradient operator from `RqElemBlock(...)`
- E_K is the face lifting operator from `RqFaceBlock(...)`
- S_K^q is the wave-only source contribution when `common.wave == 1`

Therefore the exact local derivative is

$$\frac{\partial Q_K}{\partial U_K} = M_K^{-1}\left(C_K - E_K \frac{\partial \hat{U}_K}{\partial U_K}\right),$$

plus any wave-specific term if active.

In the current backend, this derivative is not assembled explicitly as a matrix in the LDG path. Instead it is applied as a tangent operator by:

- `GetdQ(...)`
- `dRqElem(...)`
- `dRqFace(...)`
- `ApplyMinv(...)`

in `backend/Discretization/residual.cpp`.

So `GetdQ(...)` is exactly the chain-rule implementation of dQ/dU.

F. Contribution from `GetW(...)`

If `common.ncw > 0`, `GetW(...)` in `backend/Discretization/residual.cpp` computes the auxiliary field `wdg`.

This can happen through:

- explicit time/update formulas
- Newton solve of `Eos(w,u)=0`
- sourcew/DAE updates

Therefore the residual also contains a chain rule term

$$\frac{\partial \widetilde{R}_K}{\partial W_K}\frac{\partial W_K}{\partial U_K}.$$

This term is conceptually required in the exact block diagonal.

However, the current AD tangent path in `dRuResidual(...)` contains:

- `// TODO: DAE functionality not implemented yet`

So for cases with nontrivial w evolution, the exactness of the AD-based block assembly depends on the specific `GetW(...)` branch. This is an important implementation limitation.

G. Contribution from `RuElem(...)`

`RuElemBlock(...)` in `backend/Discretization/uresidual.cpp` computes the element-volume contribution.

At Gauss points it evaluates:

- time contribution
- `SourceDriver(...)`
- `FluxDriver(...)`

then applies geometry and basis testing through:

- `ApplyXx4(...)`
- `Gauss2Node(...)`

So the elemental Jacobian contribution is

$$\frac{\partial \widetilde{R}_K^{\mathrm{elem}}}{\partial U_K} = \frac{\partial \widetilde{R}_K^{\mathrm{elem}}}{\partial U_K}\Big|_{\mathrm{direct}} + \frac{\partial \widetilde{R}_K^{\mathrm{elem}}}{\partial Q_K}\frac{\partial Q_K}{\partial U_K} + \frac{\partial \widetilde{R}_K^{\mathrm{elem}}}{\partial W_K}\frac{\partial W_K}{\partial U_K} + \frac{\partial \widetilde{R}_K^{\mathrm{elem}}}{\partial AV_K}\frac{\partial AV_K}{\partial U_K}.$$

In the current backend, this total directional derivative is implemented by:

- `dRuElemBlock(...)`

which consumes:

- `sol.dudg`, already containing `dq` inserted by `GetdQ(...)`
- `sol.dwdg` if available
- `sol.dodg` if AV is active

So `dRuElemBlock(...)` is already the exact elemental Jacobian action needed for each diagonal block column.

H. Contribution from `RuFace(...)`

`RuFaceBlock(...)` in `backend/Discretization/uresidual.cpp` computes the face contribution by:

- gathering `uh`, `udg`, `wdg` on faces
- interpolating to face Gauss points
- evaluating `FhatDriver(...)` on interior faces
- evaluating `FbouDriver(...)` on boundary faces
- applying `ApplyJacFhat(...)`
- projecting back by `Gauss2Node(...)`

Therefore the face Jacobian contribution for the diagonal block is

$$\sum_{f\subset\partial K}\left(\frac{\partial\widetilde{R}_{Kf}}{\partial U_{K,f}}+\frac{\partial\widetilde{R}_{Kf}}{\partial Q_{K,f}}\frac{\partial Q_{K,f}}{\partial U_K}+\frac{\partial\widetilde{R}_{Kf}}{\partial W_{K,f}}\frac{\partial W_{K,f}}{\partial U_K}+\frac{\partial\widetilde{R}_{Kf}}{\partial\hat{U}_f}\frac{\partial\hat{U}_f}{\partial U_K}\right).$$

This total derivative is implemented by:

- `dRuFaceBlock(...)`

which uses:

- `sol.duh`
- `sol.dudg`
- `sol.dwdg`

So again, the exact diagonal-block action is already available through the tangent residual path.

I. Why the exact diagonal block should be assembled from `dResidual(...)`

This is the most important implementation decision.

Because LDG in Exasim recomputes:

- `uh`
- `q`
- `w`
- AV

inside `Residual(...)`, the exact diagonal block is not just the local derivative of `RuElemBlock(...)` with respect to the local nodal u.

It must include all implicit dependencies created by the preprocessing steps inside `Residual(...)`.

Therefore, if the goal is the exact block diagonal of the Jacobian of the actual residual returned by `Residual(...)`, the safest implementation is:

- seed a local basis vector in u
- run `dResidual(...)` or `MatVec(...)`
- extract only the same-element residual rows

That is exact by construction for the implemented residual.

4. Proposed implementation

A. Affected files

The implementation should be built around these files:

- `backend/Discretization/residual.cpp`
- `backend/Discretization/qresidual.cpp`
- `backend/Discretization/uresidual.cpp`
- `backend/Discretization/getuhat.cpp`
- `backend/Discretization/matvec.cpp`

B. Data structure for the block-diagonal matrix

Let

$$n_{\mathrm{loc}} = npe \cdot ncu.$$

Store the diagonal blocks as a dense batched array:

$$BJ \in \mathbb{R}^{n_{\mathrm{loc}} \times n_{\mathrm{loc}} \times ne1},$$

where `ne1` is the number of owned elements whose residual rows are in `res.Ru`.
In code terms this should be a flat array of length:

- `nloc * nloc * common.ne1`

stored element-block by element-block, column-major within each block so it is immediately compatible with batched BLAS/LAPACK-style inversion.

C. Exact assembly algorithm using `dResidual(...)`

This is the recommended exact implementation when Enzyme AD is available.
For each owned element K:

1. Define local block size
$$n_{\mathrm{loc}} = npe \cdot ncu.$$
2. For each local column $j = 0, \ldots, n_{\mathrm{loc}} - 1$:
   - zero all tangent fields:
     - `sol.dudg`
     - `sol.dwdg` if used
     - `sol.duh`
     - `sol.dodg` if AV is active
     - `res.dRu`
     - `res.dRq`
     - `res.dRh`

- place a unit seed in `sol.dudg` corresponding to the j-th `u` DOF of element K
- call:
  - `dResidual(sol, res, app, driver_abi, master, mesh, tmp, common, handle, backend)`
- extract the element-residual slice of `res.dRu` corresponding to element K
- store that slice as column j of the dense block J_{KK}

This works because `dResidual(...)` already performs:

- `GetdUhat(...)`
- `GetdQ(...)`
- `GetdAv(...)`
- `dRuElem(...)`
- `dRuFace(...)`

and applies the final residual scaling.
So each call returns exactly:

$$J(U)\, e^{(K,j)},$$

and extracting the rows of element K gives precisely the column j of J_{KK}.

D. Exact assembly algorithm using `MatVec(...)`

If Enzyme AD is not available, use the same column-by-column procedure but replace `dResidual(...)` with:

- `MatVec(...)`

from `backend/Discretization/matvec.cpp`.
For each seed vector $e^{(K,j)}$:

- `MatVec(...)` returns $J(U)e^{(K,j)}$ using either:
  - first-order finite difference
  - second-order finite difference
- then extract the same-element residual rows and store the column

This is more expensive and approximate, but it differentiates the actual `Residual(...)` pipeline and therefore includes all implemented couplings.

E. Practical column-seeding detail

The seed must be inserted only into the u components of `sol.dudg`, not into q or other fields.

That means:

- within element K
- within the `udg` layout
- only the first `ncu` components per node are seeded

All q-component tangent entries must start from zero and be generated by `GetdQ(...)`.

This is essential. Otherwise the result is not $\partial R/\partial U$, but a derivative with independently perturbed q.

F. Optional inversion for block-Jacobi use

If the goal is a block-Jacobi preconditioner, after assembling the blocks one can invert them batched:

$$J_{KK}^{-1}, \qquad K = 1, \ldots, ne1.$$

This can reuse the same batched inversion pattern already used elsewhere in Exasim for dense small blocks.

The natural storage layout is chosen specifically so that a single batched inversion call can process all element blocks.

G. Why this is better than hand-assembling a separate LDG Jacobian first

A manual Jacobian assembly would need to reproduce every chain-rule dependency already encoded in `Residual(...)`, including:

- interior/boundary `uh` construction
- local q-recovery
- local w-recovery
- AV differentiation
- face orientation details
- sign/time scaling

That is possible, but it is much more invasive and easier to get wrong.

Using `dResidual(...)` or `MatVec(...)` avoids duplicating solver logic and guarantees consistency with the actual residual definition.

5. Risks or numerical concerns

- The AD path is not fully general for all w/DAE branches. In `backend/Discretization/residual.cpp`, `dRuResidual(...)` explicitly notes:
  - `TODO: DAE functionality not implemented yet`
- Therefore:
  - AD-based block assembly is exact only for the branches actually covered by the tangent implementation
  - FD-based `MatVec(...)` is more complete, but approximate
- Finite-difference block assembly is expensive:
  - n_{loc} residual evaluations per element block
  - or one global `MatVec(...)` per local column if done naively
- Finite-difference quality depends on:
  - `common.matvecTol`
  - `common.matvecOrder`
- For interface elements in MPI, the exact diagonal block still requires the MPI tangent/residual path, because face terms depend on exchanged neighbor states even when only the diagonal rows are kept.
- If AV is active and unfrozen, the exact block diagonal of the residual must include `GetAv(...)` differentiation. Omitting that gives a different Jacobian.
- A hand-derived "element-only" approximation will generally miss orientation-dependent face-trace effects from `GetUhat(...)`.

6. Verification plan

1. **Column test against `dResidual(...)`**
   - Pick an element K and a random local vector z_K.
   - Form a global seed z supported only on element K.
   - Compute $y = Jz$ with `dResidual(...)`.
   - Extract the element-K rows of y.
   - Compare with $J_{KK} z_K$ from the assembled dense block.
2. **Finite-difference spot check**
   - For a few local columns j, compare the assembled column against:

$$\frac{R(U + \varepsilon e^{(K,j)}) - R(U)}{\varepsilon}$$

     restricted to the element-K residual rows.

3. **Boundary and interior element checks**
   - Test one boundary element and one interior element.
   - This verifies both:
     - `FbouDriver(...)`
     - `FhatDriver(...)`
     - `UbouDriver(...)`
4. **Physics toggles**
   - Verify separately for:
     - `common.tdep = 0`
     - `common.tdep = 1`
     - `common.ncq = 0` and `> 0`
     - `common.ncw = 0` and a case with `ncw > 0`
     - AV frozen and unfrozen
5. **Residual-consistency check**
   - Confirm the assembled block includes the final residual scaling by `-1` and `1/common.dtfactor` exactly as in `Residual(...)`.
6. **Performance sanity**
   - Measure cost per element block.
   - If explicit block assembly is too expensive, keep the same mathematical definition but consider:
     - assembling blocks only when the nonlinear state changes significantly
     - or using the block assembly only for preconditioner setup, not every Krylov iteration